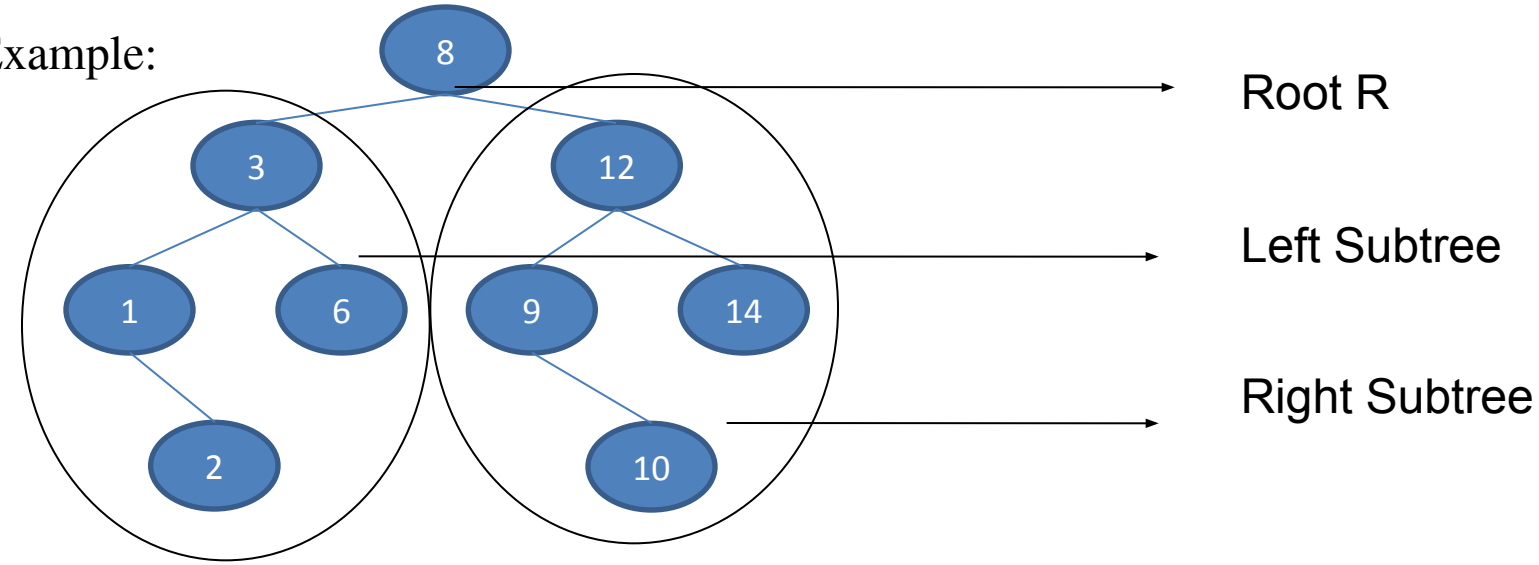
Data Structure
MTHR2207

Md. Manowarul Islam
Dept. of CSE, Jagannath University

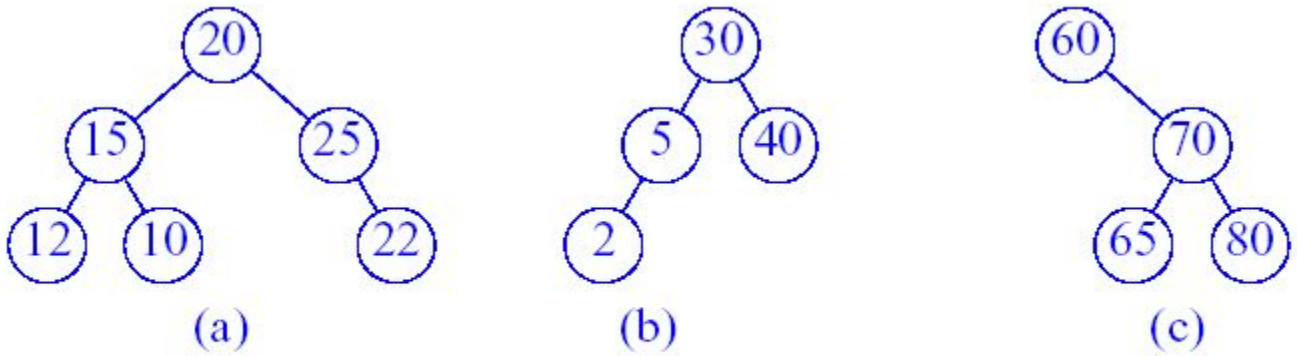
Binary Search Tree (BST)

- It is a hierarchical, nonlinear data structure.
- Its properties are given bellow:
  - Each node has a value.
  - A total order is defined on these node values.
  - Left subtree of a node contains the values less than the node's value.
  - Right subtree of a node contains the values greater than or equal to the node's value.

Example:



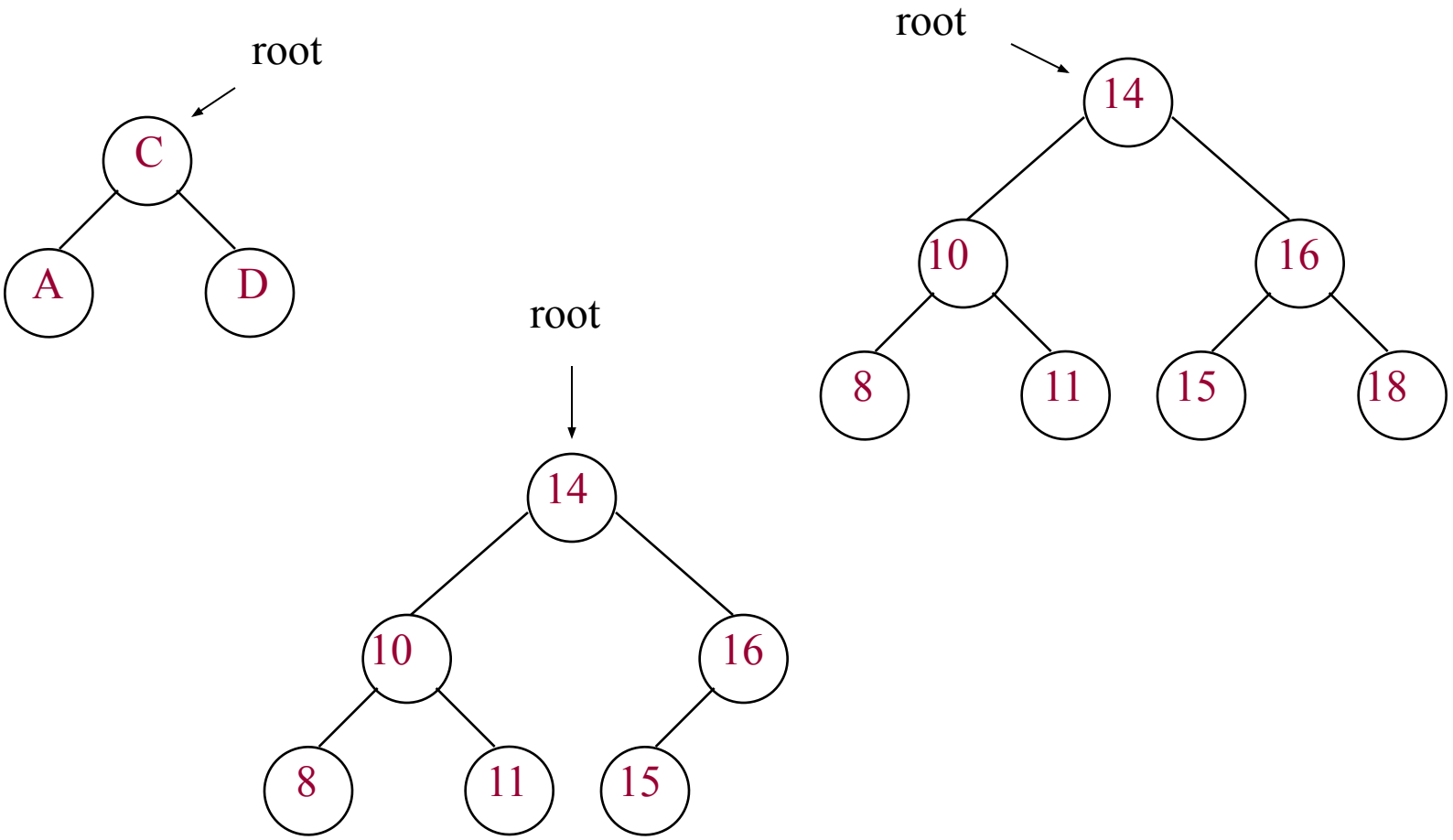
Examples of Binary Trees



Binary Trees

- Which of the above trees are binary search trees?
  - (b) and (c)
- Why isn't (a) a binary search tree?
  - It violates the property #3

Binary Search Trees: Examples



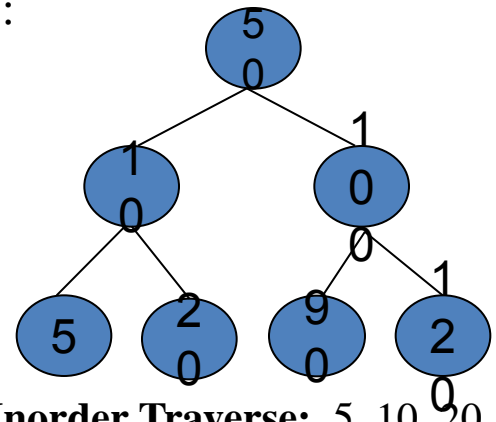
Sorting of A List of Items Using BST

- A binary search tree can be used to implement a simple but inefficient sorting algorithm.
- Two steps are followed for sorting:
  - TreeNode := Create_BST(List of Items)
  - Traverse_InOrder(TreeNode)

- Example:

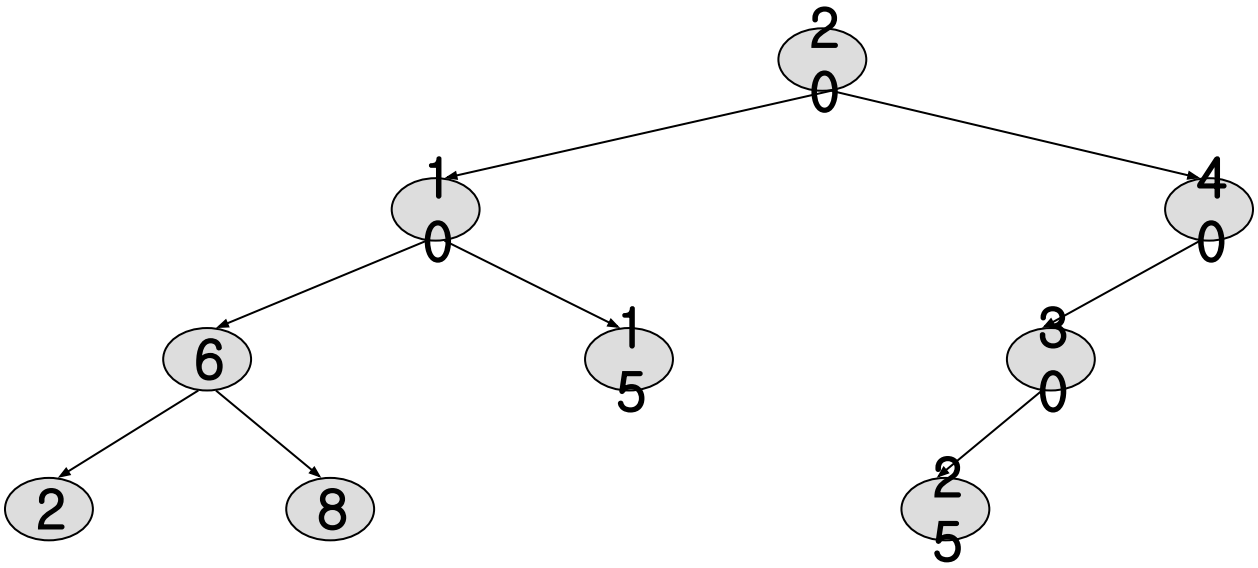
List of Items: 50, 10, 5, 100, 120, 90, 20

BST:



Inorder Traverse: 5, 10, 20, 50, 90, 100, 120 [Sorted List]

The Operation Ascend()



- How can we output all elements in ascending order of keys?
  - Do an inorder traversal (left, root, right).
- What would be the output?
  - 2, 6, 8, 10, 15, 20, 25, 30, 40

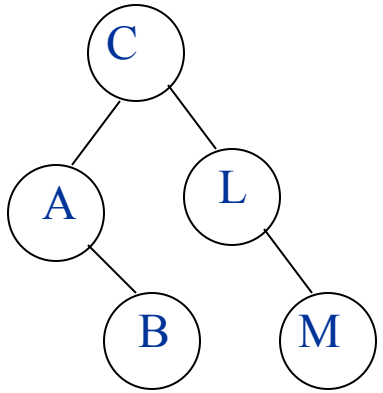
Sorting with a BST

Given a BST can you output its keys in sorted order?

Visit keys with Inorder:

- visit left
- print root
- visit right

Example:



Inorder visit prints:

A B C L M

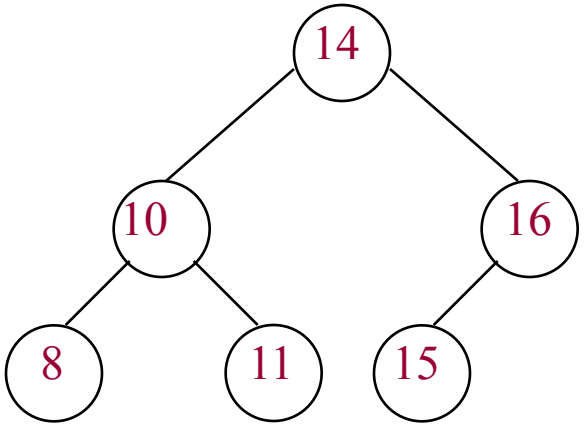
Searching for a key in a BST

compare x to key of root:

- if x = key return
- if x < key => search in L
- if x > key => search in R

search L or R in the exact same way

Example:



x=8 is ??? X=17 is ???

Searching a node with a specific value from BST Rules

- Examine the root R.
- If the searching value equals the root, then the value exists in the tree.
- If it is less than the root, then it must be in the left subtree.
- If it is greater than the root, then it must be in the right subtree.
- This process continues until the searching value is found or a leaf node is reached without having that value where it would be.

Algorithm

- **Search_BST(Root, Left, Right, Key)**

1. Loc := Search(Root, Key, NULL)
2. If Loc = NULL then Print: "Item not in BST".
3. Else Print: "Item is in BST".
4. End.

Here,

Root = Root of BST

Left = Left Subtree

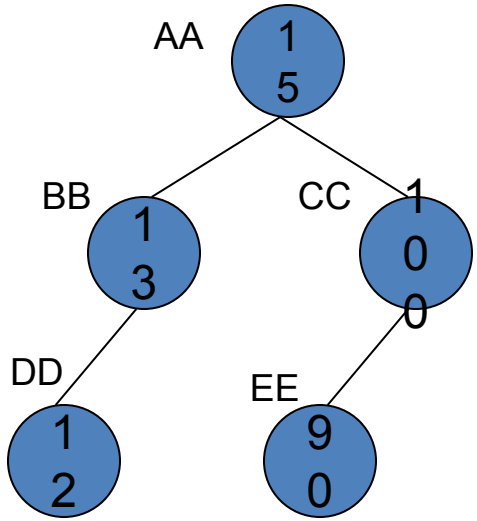
Right = Right Subtree

Key = Searching Item

- **Search(Node, Key, Par)**

1. If Node = NULL then return NULL.
2. If Key < Node.Key then Search(Node.Left, Key, Node).
3. Else if Key > Node.Key then Search(Node.Right, Key, Node).
4. Else return Node

Example:



Searching Item: 12

Steps:

1. Search(Root, 12) → Search(AA,12)
2. Search(AA->Left,12) → Search(BB, 12)
3. Search(BB->Left, 12) → Search(DD, 12)
4. Return DD and Print: Item is in BST.

Inserting a Node into a BST

Rules

- Examine the root R.
- If the new value is less than the root's, then it must be inserted at the left subtree.
- Otherwise it must be inserted at the right subtree.
- This process continues until the new value is compared with a leaf node and then it is add as that node's (leaf node) left or right child depending on its value.

Algorithm

- **Insert_BST(Root, Left, Right, Item)**

1. Set NewNode.Value := Item.
2. Insert(Root, NewNode).

End.

Here,

Root = Root of BST

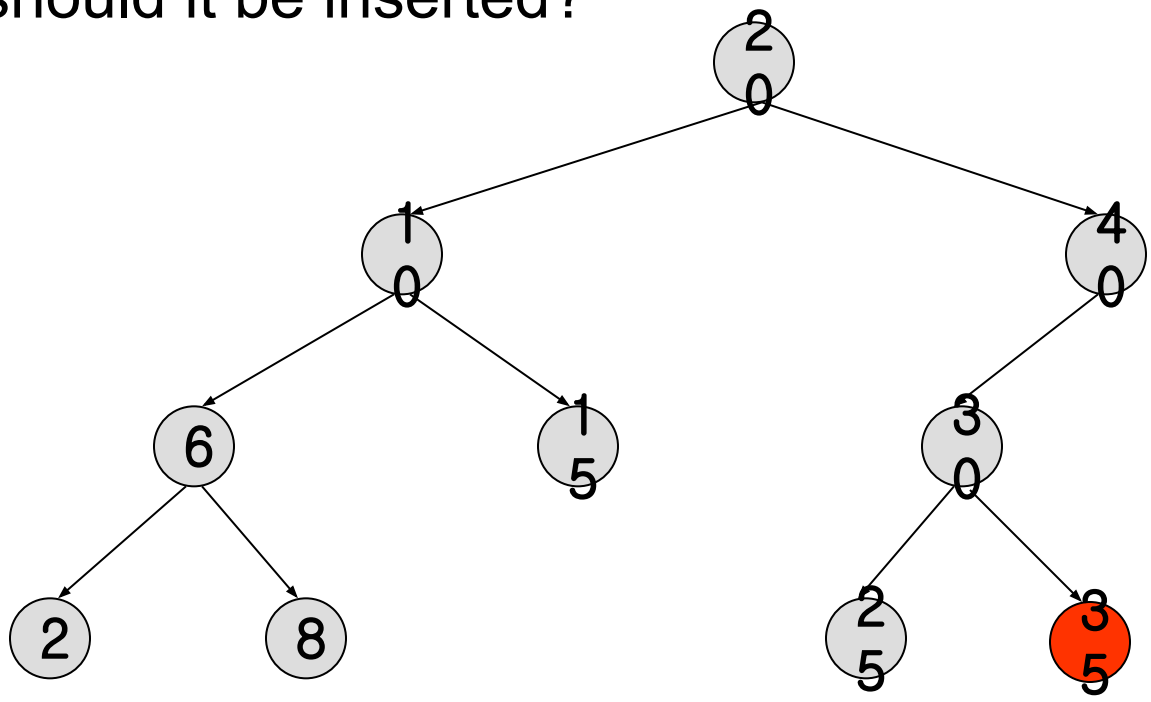
Left = Left Subtree

Right = Right Subtree

Item = New Item to be inserted

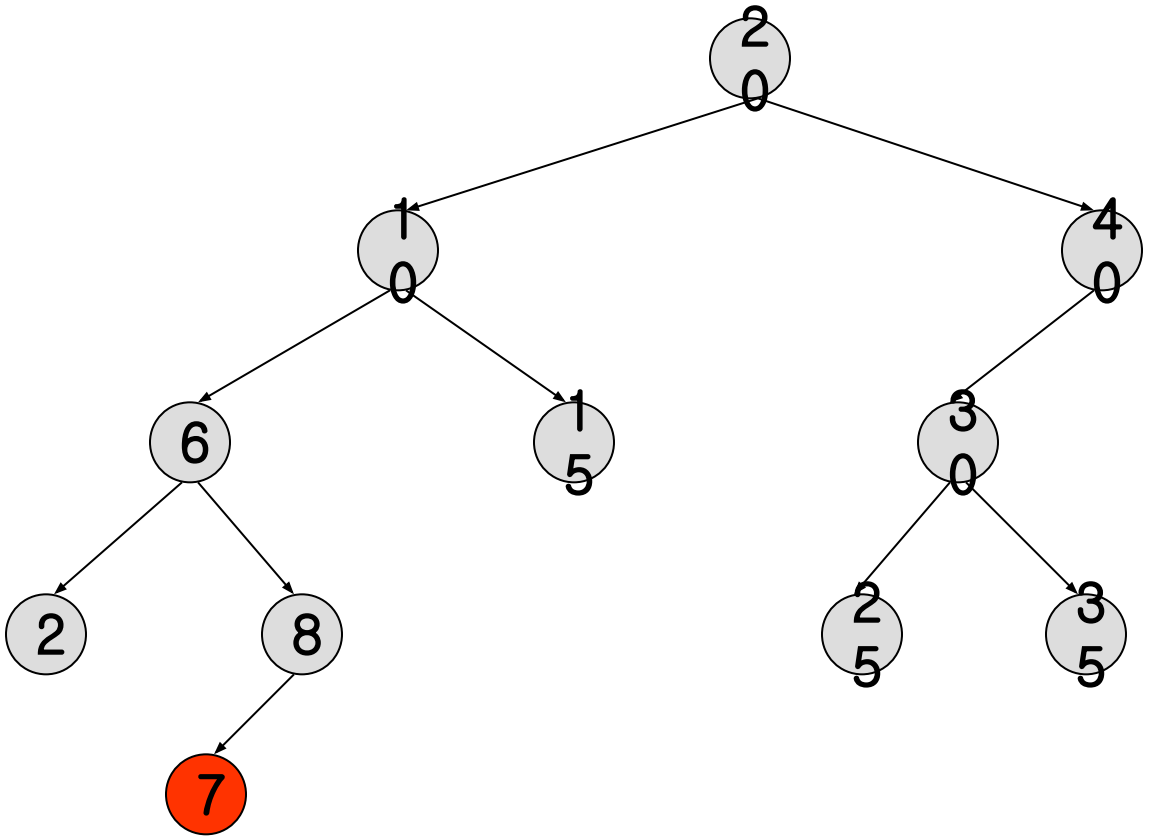
Insert Example

We wish to insert an element with the key 35.
Where should it be inserted?



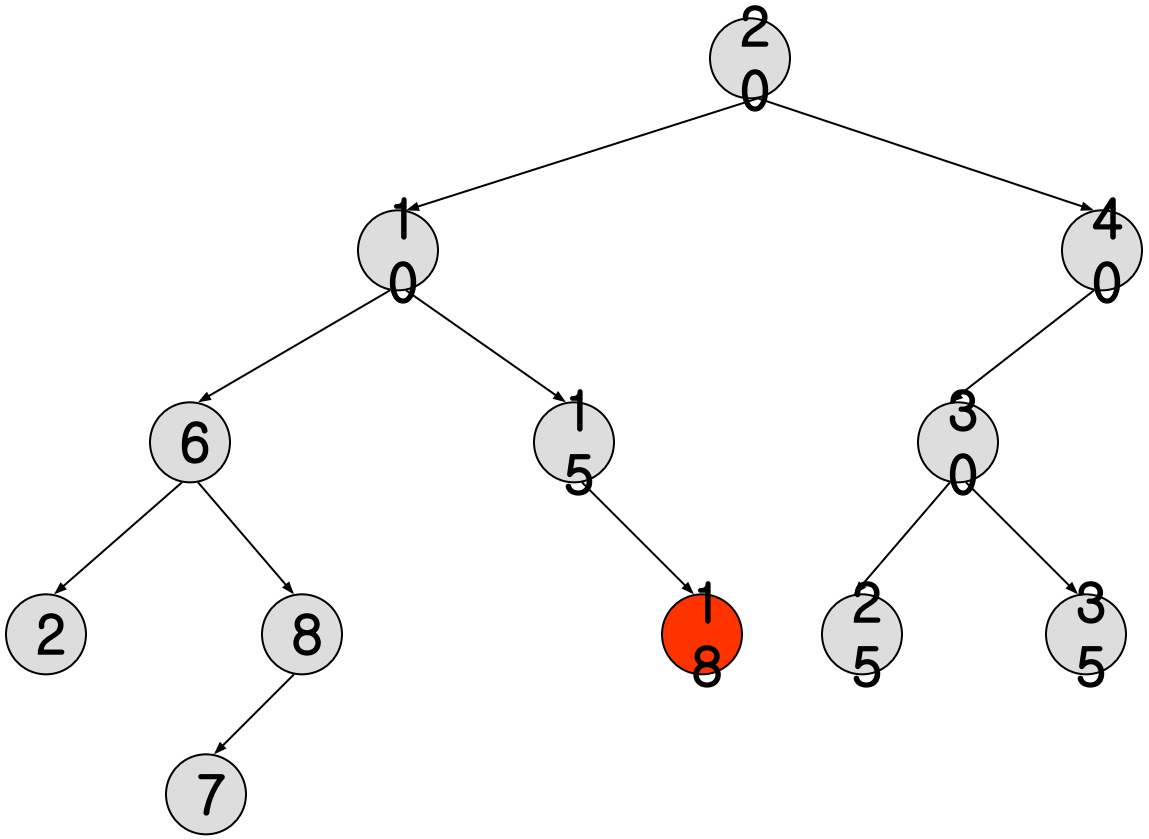
Insert Example

Insert an element with the key 7.



Insert Example

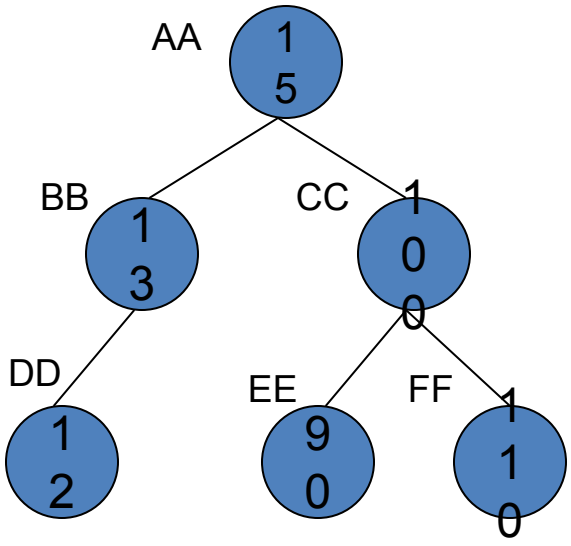
Insert an element with the key 18.



Insert(Node, NewNode)

1. If Node = NULL then Node := NewNode and return.
2. If NewNode.Value < Node.Value then Insert(Node.Left, NewNode).
3. Else Insert(Node.Right, NewNode).

Example:



New Item: 110

FF.Value=110 (FF is the New Node address)

Steps:

1. Insert(Root, FF) → Insert(AA, FF)
2. Insert(AA->Right, FF) → Insert(CC, FF)
3. Insert(CC->Right, FF)

Figure: BST with 5 Nodes

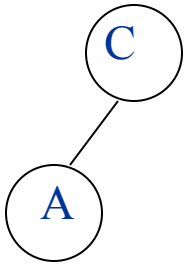
Building a BST

Build a BST from a sequence of nodes read one a time

Example: Inserting **C A B L M** **(in this order!)**

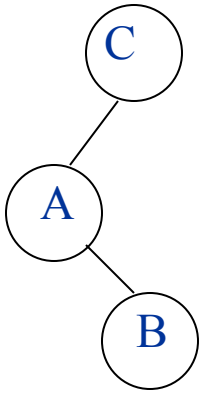
1) Insert C

2) Insert A

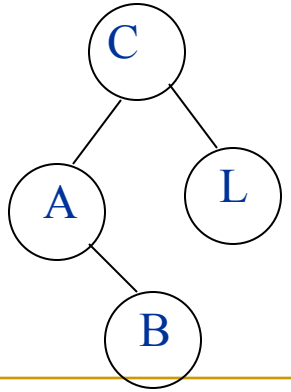


Building a BST

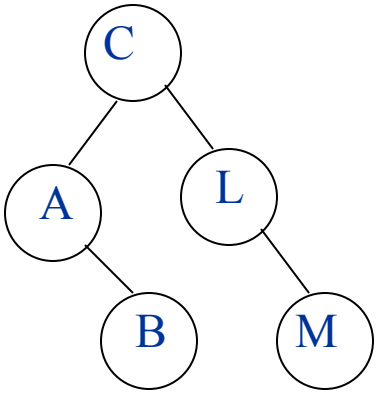
3) Insert B



4) Insert L



5) Insert M

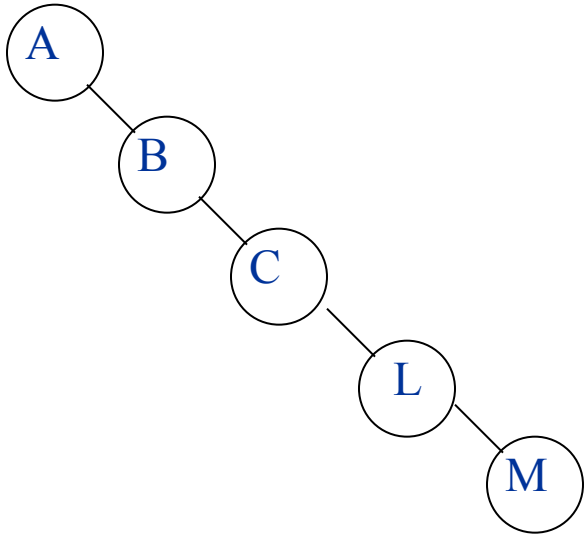


Building a BST

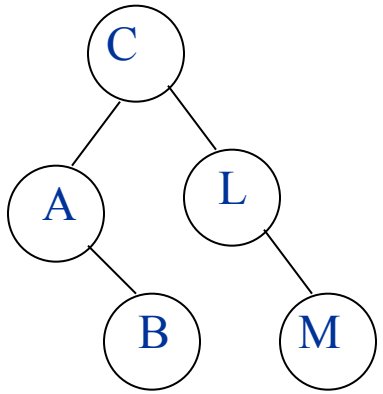
Is there a unique BST for letters A B C L M ?

NO! Different input sequences result in different trees

Inserting: A B C L M



Inserting: C A B L M

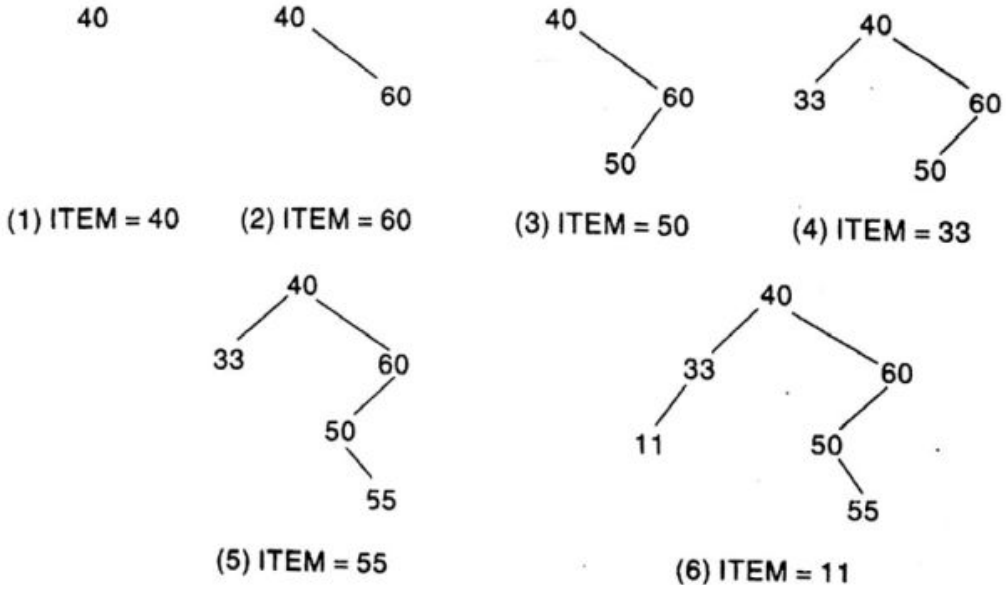


Building a BST

Suppose the following six numbers are inserted in order into an empty binary search tree:

40, 60, 50, 33, 55, 11

Figure 7.23 shows the six stages of the tree. We emphasize that if the six numbers were given in a different order, then the tree might be different and we might have a different depth.



Deleting a Node from BST

Rules

- A node N has no children. N is deleted from BST by simply replacing the location of N in the parent node P(N) by the NULL pointer.
- A node N has one child. N is deleted from BST by simply replacing the location of N in the parent node P(N) by the location of the only child of N.
- A node N has two children. Let S(N) denote the inorder successor of N. N is deleted from BST by first deleting S(N) from BST and then replacing node N by S(N).

Example:

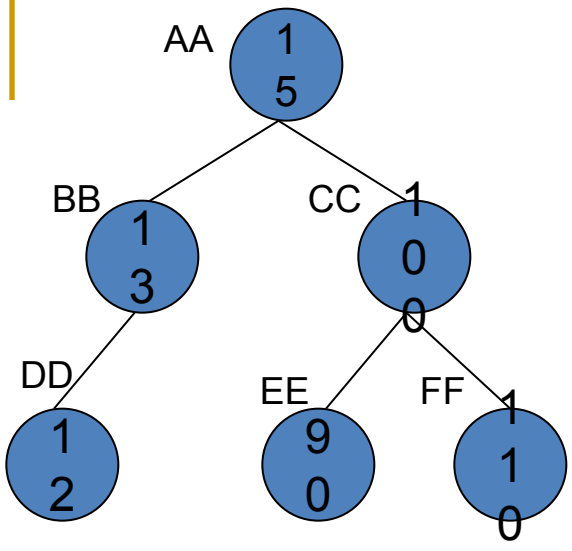


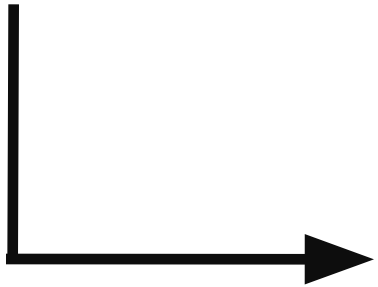
Figure: Given BST

Deleted Item: 15

Loc = AA and Par = NULL

Steps:

1. Delete_A(Root, Loc, Par)
2. Inorder Successor, S(Loc) = EE
3. Delete EE from BST
4. Replace AA by EE by EE.Left = AA.Left and EE.Right := AA.Right.



After Delete Operation

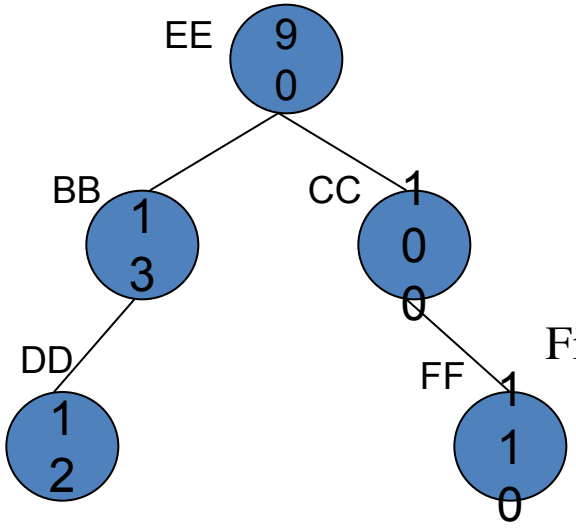


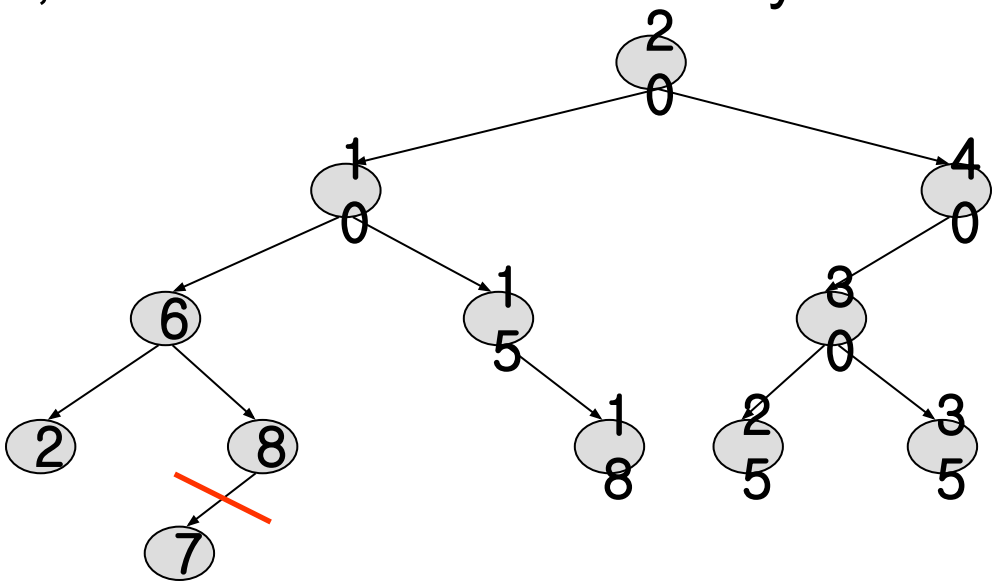
Figure: Resulting BST

The Operation Delete(key, e)

- For deletion, there are three cases for the element to be deleted:
  1. Element is in a leaf.
  2. Element is in a degree 1 node (i.e., has exactly one nonempty subtree).
  3. Element is in a degree 2 node (i.e., has exactly two nonempty subtrees).

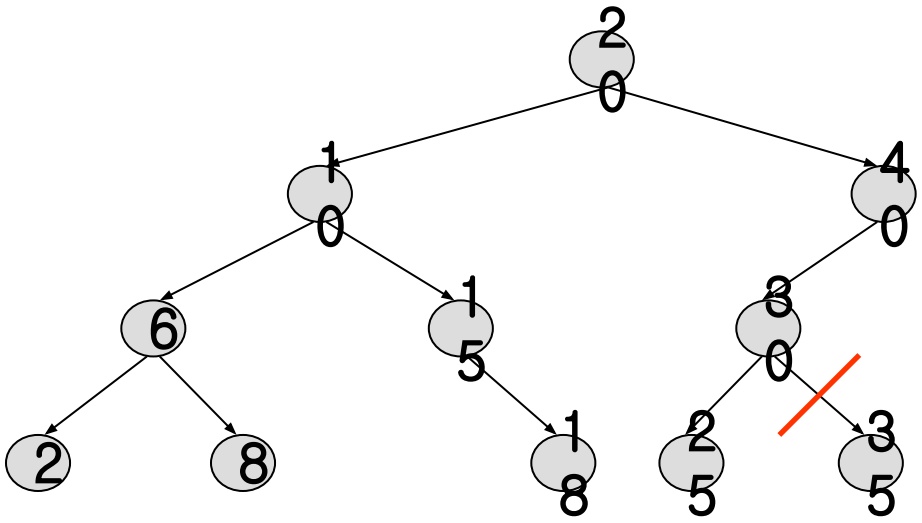
Case 1: Delete from a Leaf

- For case 1, we can simply discard the leaf node.
- Example, delete a leaf element. key=7

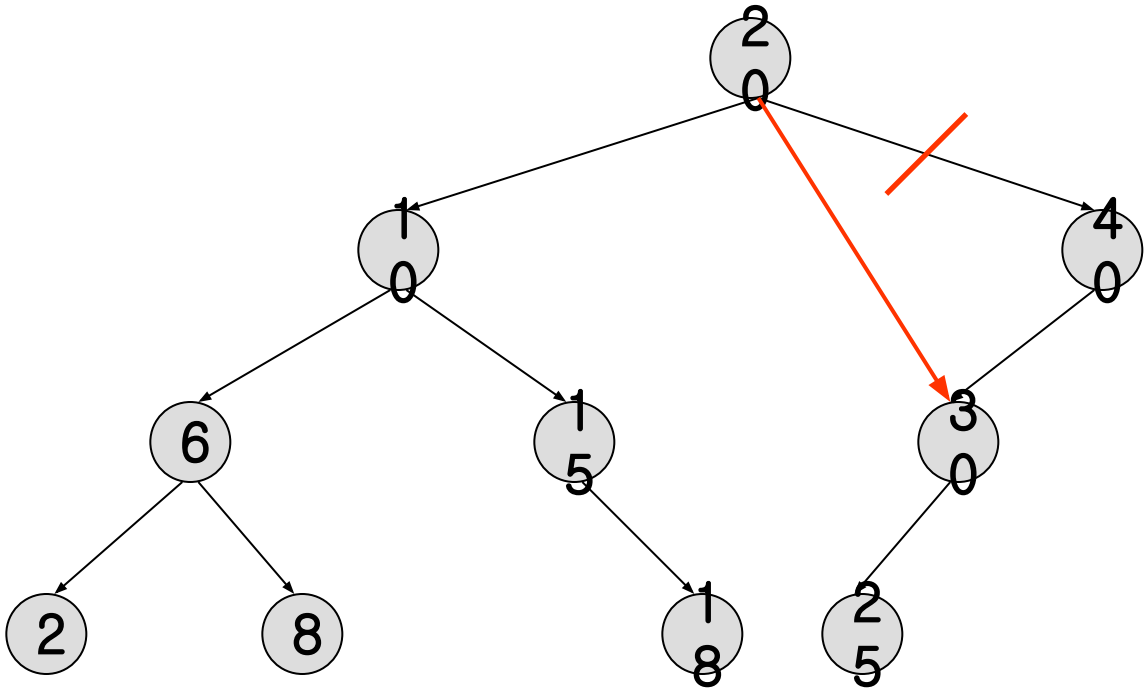


Case 1: Delete from a Leaf

Delete a leaf element. key=35



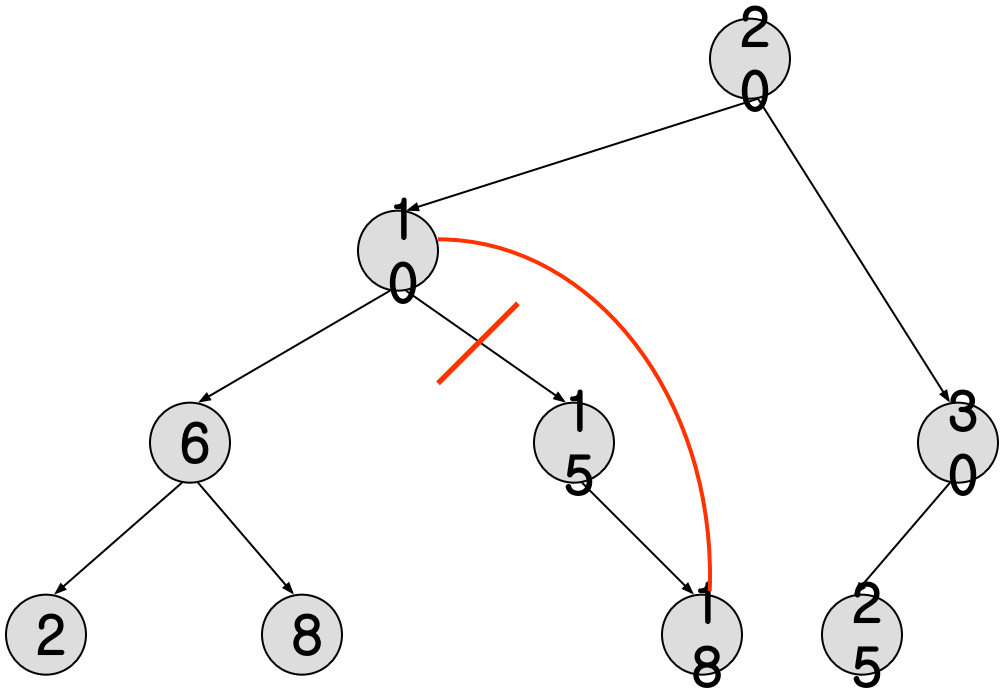
Case 2: Delete from a Degree 1 Node



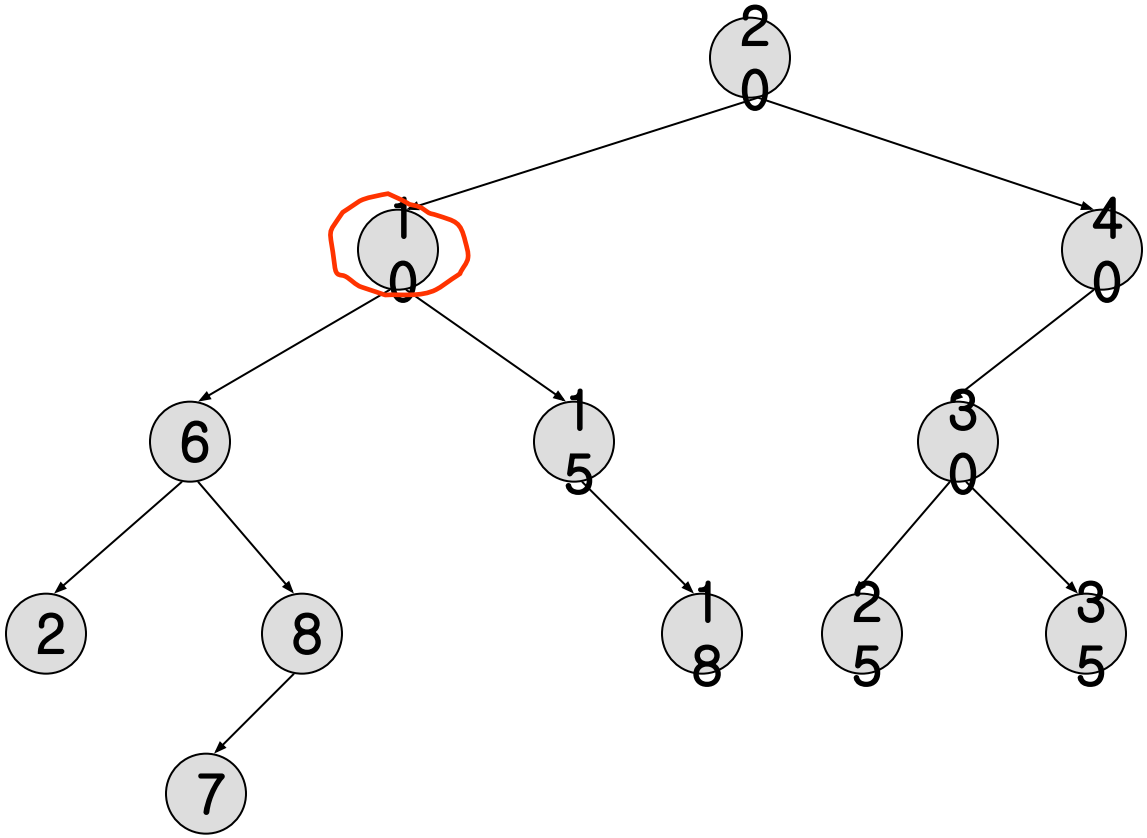
- Which nodes have a degree 1?
- Example: Delete key=40

Case 2: Delete from a Degree 1 Node

Delete from a degree 1 node. key=15

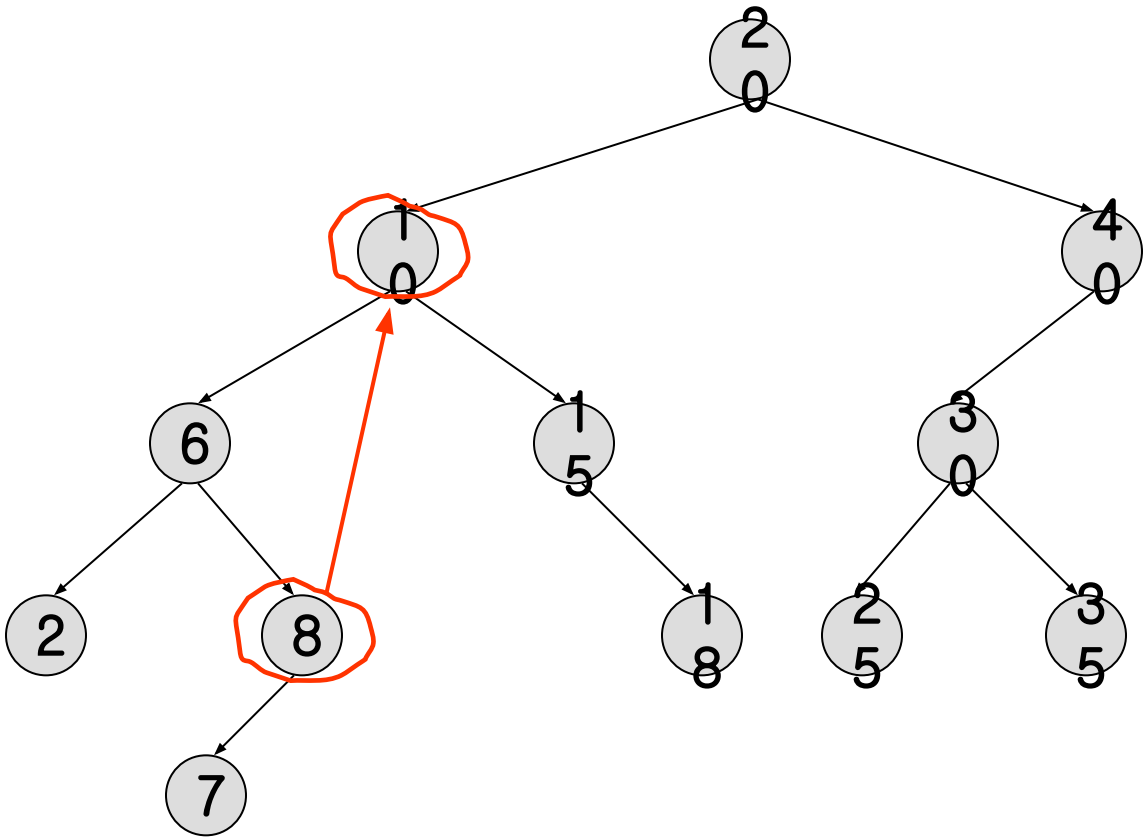


Case 3: Delete from a Degree 2 Node



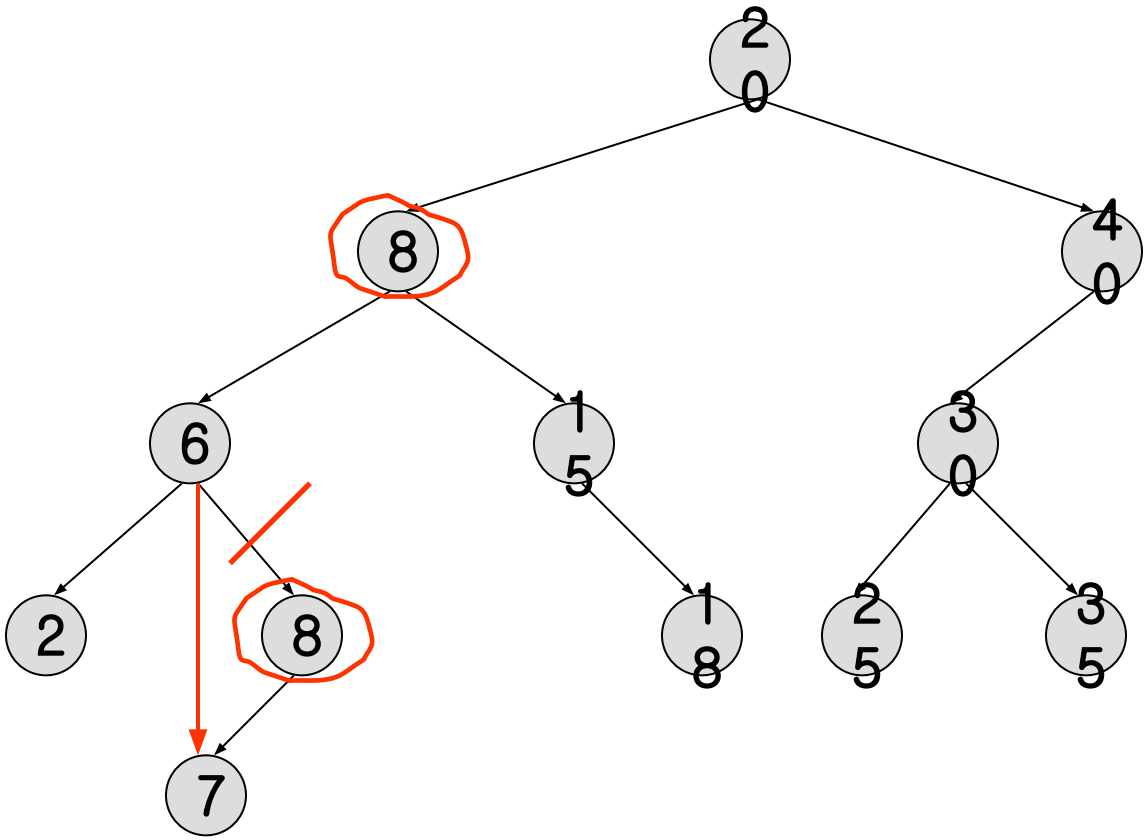
- Which nodes have a degree 2?
- Example: Delete key=10

Case 3: Delete from a Degree 2 Node



- Replace with the largest key in the left subtree (or the smallest in the right subtree)
- Which node is the largest key in the left subtree?

Case 3: Delete from a Degree 2 Node



The largest key must be in a leaf or degree 1 node.

Case 3: Delete from a Degree 2 Node

- Note that the node with largest key in the left subtree (as well as that with smallest in the right subtree) is guaranteed to be in a node with either zero or one nonempty subtree
- How can we find the node with **largest key** in the left subtree of a node?

  - by moving to the root of that subtree and then following a sequence of right-child pointers until we reach a node whose right-child pointer is NULL
- How can we find the node with **smallest key** in the right subtree of a node?

  - by moving to the root of that subtree and then following a sequence of left-child pointers until we reach a node whose left-child pointer is NULL

Example:

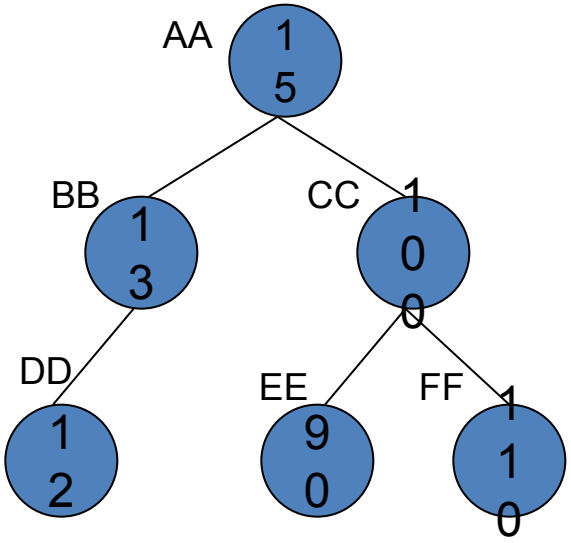


Figure: Binary Search Tree

Inorder Traversal: 12, 13, 15, 90, 100, 110

Preorder Traversal : 15, 13, 12, 100, 90, 110

Postorder Traversal : 12, 13, 90, 110, 100, 15

End